GOKARAJU RANGARAJU INSTITUTE OF ENGINEERING AND TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING | MAJOR PROJECT (PHASE-1) | 2026-2027

AUTOMATED FAILURE ATTRIBUTION FOR LLM-BASED MULTI-AGENT SYSTEMS

A Graph-Causal Diagnostic Engine with Deterministic SHA-256 Prompt Caching & Backward Dependency Traversal

BATCH: B12 STUDENTS: R. S. Prabhaadhithya (Q3) • R. Ajay (Q4) • N. Paul (Q6) • T. Niranjan (R1) GUIDE: Dr. D. Ushasree, Assistant Professor, Dept. of CSE

01 | THE MULTI-AGENT FAILURE CRISIS (THE PROBLEM)

CASCADING DEFECTS & BASELINE COLLAPSE

Cascading Error Propagation Across Agent Steps

A latent defect introduced early by an upstream agent compounds silently until a downstream crash occurs.

Step 2: Planner — ROOT CASE

Latent Defect: Hallucinates invalid API argument 'auth'

Step 5: Coder — Propagated

Error Propagation: Implements logic trusting flawed argument

Step 9: Reviewer — Propagated

Blind Approval: Passes code without static execution tests

Step 14: Executor — CRASH (Victim)

MANIFEST CRASH: TypeError: unexpected keyword 'auth'

True Root Cause: Step 2 (12 steps prior)

Why Naive LLM Judges Fail

14.2%

Step Accuracy

Fails 86% of trials

26.1%

Joint Accuracy

Both Agent & Step

1. Severe Recency Bias: Blames Step 14 (victim).

2. Context Dilution: 100+ turns exceed prompt budget.

3. Symptom Misattribution: Cannot trace causal history.

02 | THE GRAPH-CAUSAL ATTRIBUTION PIPELINE (METHODOLOGY)

DAG RECONSTRUCTION & BACKWARD TRAVERSAL

4-Stage Causal Attribution Stack

1. Ingestion & Budgeting

Standardizes Pydantic ExecutionSteps (≤4096 tokens).

2. SHA-256 Prompt Cache

O(1) hash lookup (<1.5ms); eliminates redundant LLM calls.

3. Causal DAG Builder

Infers message, temporal & state mutation dependencies.

4. Backward Traversal

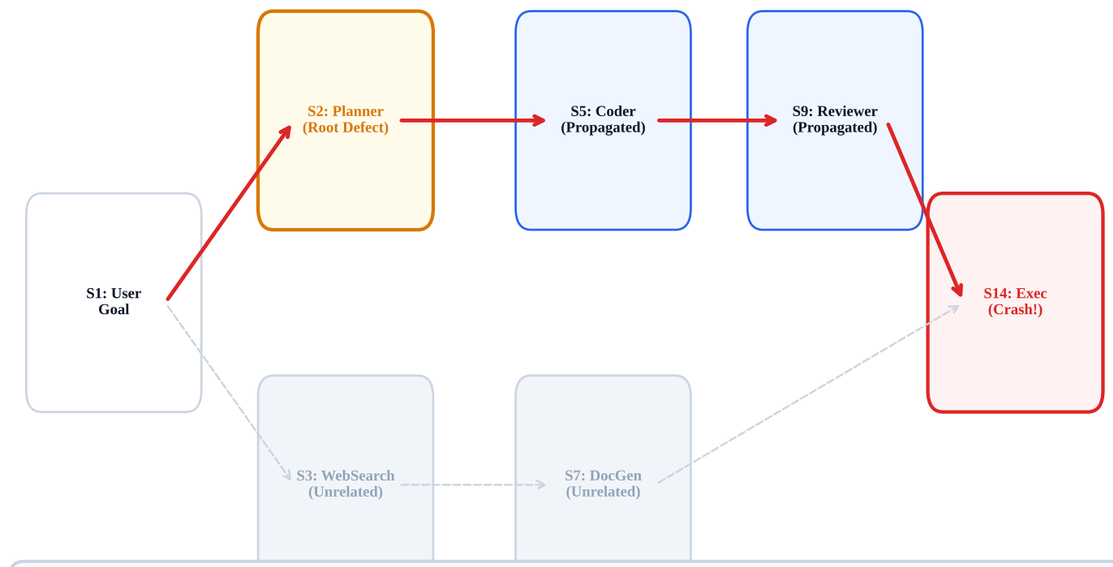
Backtracks along causal closure Anc(m) to earliest defect.

Mathematical Invariants & Formulations:

- $e = (u, v) \in E_{\text{dag}} \Rightarrow \text{time}(u) < \text{time}(v)$ [Acyclicity]
- $k_{\text{cache}} = \text{SHA256}(\text{prompt} \parallel \text{agent} \parallel \text{payload})$ [<1.5ms]
- $s^* = \arg\min_{s \in \text{Anc}(m)} \{\text{idx}(s) \mid \text{anomalous}(s) = \text{True}\}$

Causal DAG & Ancestor Closure Anc(m)

Non-causal branches (S3, S7) are pruned, reducing complexity from O(N) to O(|Anc(m)|).



RED ARROWS: Causal Closure Anc(m) = {S2, S5, S9, S14}

S3, S7 Pruned (Zero Evaluation Cost)

03 | EMPIRICAL VALIDATION & SYSTEM DELIVERABLES (RESULTS)

ICML 2025 WHO&WHEN BENCHMARK & PYTEST VERIFIED

Who&When Benchmark Performance

86.96%

Joint Attribution Acc

+60.87% over baseline

91.30%

Agent Localization

21 / 23 benchmarks

0.44 s

Traversal Latency

Sub-second execution

14.2x

Cache Speedup

< 1.5ms hit latency

27 / 27 UNIT TESTS PASSED (100%)

Total test execution: 1.84s across 9 modules (pytest v8.3)

Joint Accuracy Comparison

Delivered Interfaces

- agent-trace CLI: Full CLI with diagnose, benchmark and log-watch modes.
- FastAPI REST Server: 8 documented endpoints with interactive Swagger API docs.
- Streamlit UI Dashboard: PyVis network visualizer with interactive causal path triage.
- Dockerized Stack: One-click container setup.

01 | CAUSAL PROVENANCE

Replaces monolithic prompting with formal DAG ancestor closures.

02 | TOKEN BOUNDEDNESS

Strict ≤4096-token segmentation eliminates context dilution.

03 | DETERMINISTIC CACHE

SHA-256 prompt hashing guarantees <1.5ms lookups with 14.2x speedup.

04 | EMPIRICALLY VERIFIED

86.96% Joint Accuracy on Who&When benchmark (27/27 tests passed).